

Lab 3 – Attacchi su TCP

Sicurezza dei Sistemi e delle reti

Modulo 4 - Protocolli TCP

Unità didattica 2 – Vulnerabilità di TCP/IP

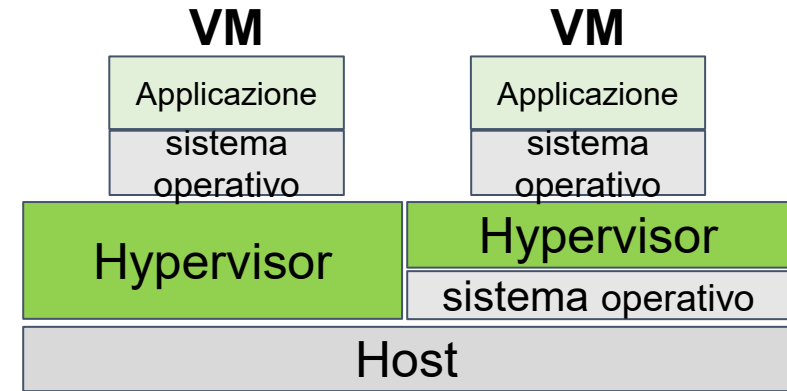
Stelvio Cimato

Esecuzione tradizionale e virtualizzata



- Fisico Ospiti eseguire un sistema operativo (ad esempio, Windows o Linux).
- Molteplici applicazioni eseguito sul sistema operativo condiviso .

¹ <https://www.virtualbox.org/>



- Un software speciale, l' **Hypervisor** , mette a disposizione le Macchine Virtuali.
- Esempi di tali tecnologie sono *Virtualbox* ¹ o *Linux KVM* ² .
- La VM è una macchina completa che esegue tutti i componenti, incluso il proprio sistema operativo, sopra l'hardware virtualizzato

²<https://www.linux-kvm.org>

Distribuzione tradizionale e virtualizzata

3

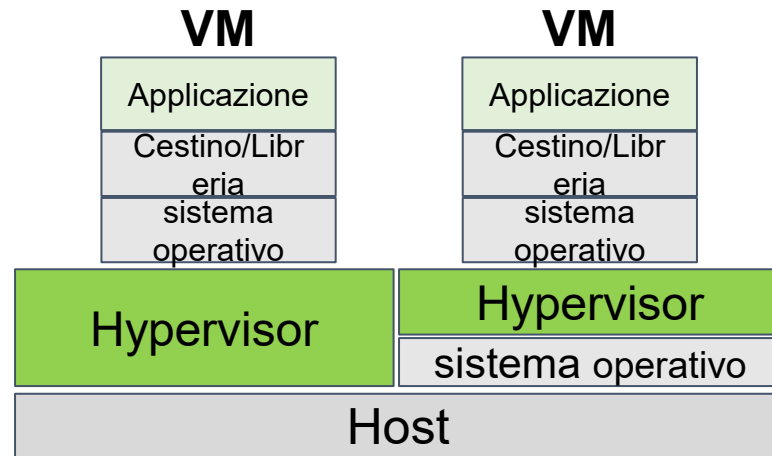
Esecuzione tradizionale

- Nessun modo per definire i limiti delle risorse per le applicazioni.
- L'isolamento delle applicazioni richiede l'esecuzione su server fisici diversi (costosi e le risorse potrebbero essere sottoutilizzate).

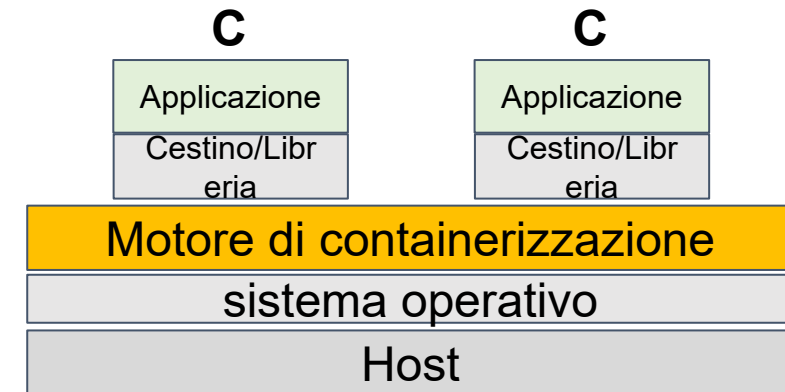
Esecuzione virtualizzata

- La virtualizzazione consente:
 - applicazioni da isolare tra le VM
 - migliore utilizzo delle risorse in un server fisico
 - migliore scalabilità.

Esecuzione virtualizzata o in contenitori



- Hardware virtuale
 - Ogni VM ha un sistema operativo e un'applicazione
 - Condividi le risorse hardware dall'host fisico



- Sistemi operativi virtuali
 - Ambienti isolati, ovvero **contenitori**, che condividono lo stesso sistema operativo *reale*
 - I contenitori vengono eseguiti da un'immagine distinta che fornisce tutti i file (Bin/, Libreria) necessari per supportarli
 - Un esempio di tali tecnologie è *Docker*¹.

¹ <https://www.docker.com/>

Distribuzione virtualizzata o in contenitori

Esecuzione virtualizzata

- Pesante: ogni VM si basa su una copia completa di un sistema operativo.
- Fornisce un isolamento completo.
- Ideale per quando si hanno applicazioni che devono essere eseguite su *diverse* versioni del sistema operativo.

Esecuzione dei contenitori

- Leggero : la condivisione delle risorse del sistema operativo riduce significativamente il sovraccarico richiesto per l'esecuzione dei contenitori.
- Fornisce un livello di processo (rilassato) . isolamento .
- Ideale per quando si hanno applicazioni che devono essere eseguite su un *singolo* kernel del sistema operativo.

Immagini e contenitori Docker

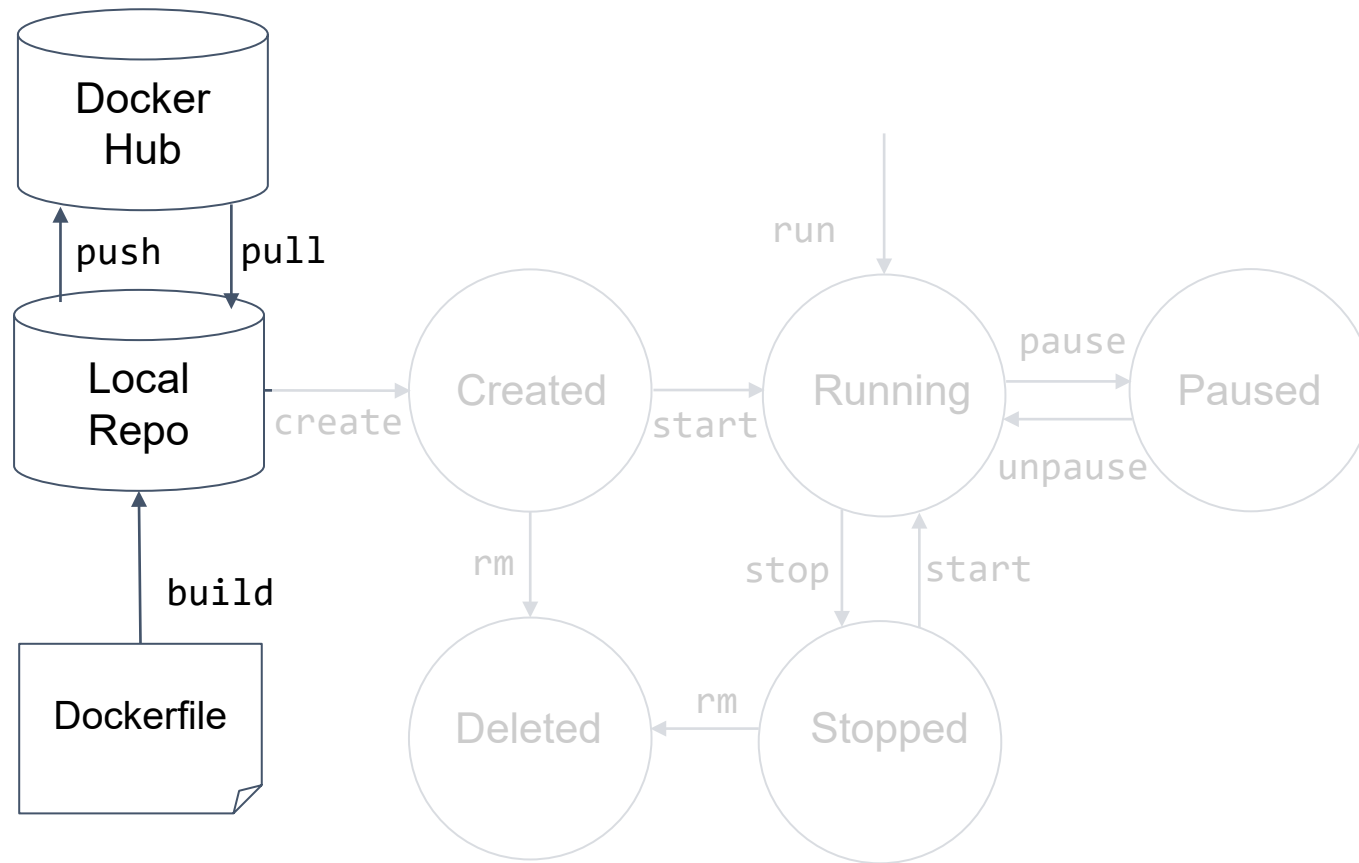
Immagine

- **immutabile** per contenitori
- Include tutto il necessario per eseguire un'applicazione
 - codice, runtime, strumenti di sistema, librerie di sistema e impostazioni

Contenitore

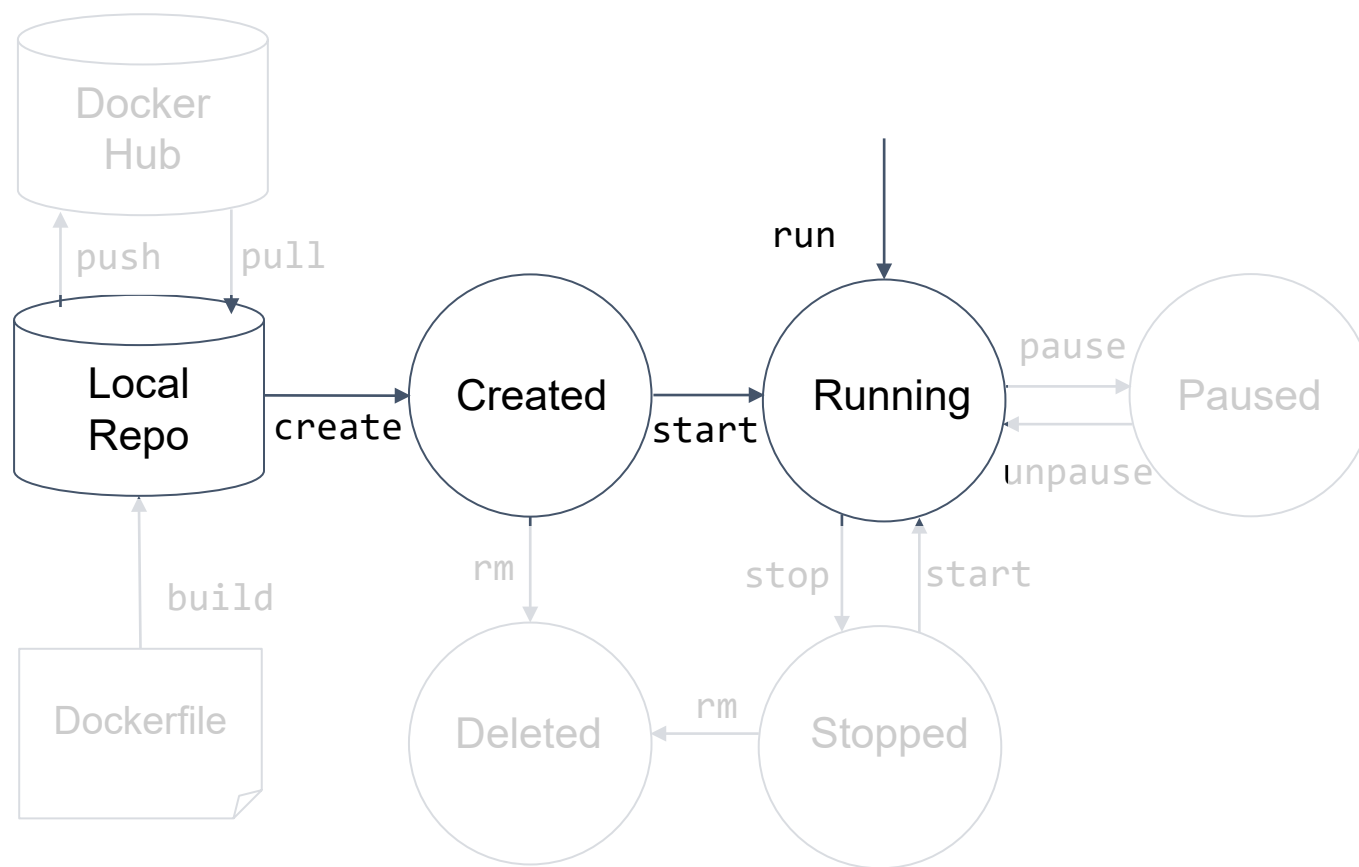
- Un'istanza di un'immagine
- Aggiungi un **nuovo livello scrivibile** sopra l'immagine sottostante
 - tutte le modifiche apportate al contenitore in esecuzione (ad esempio, la scrittura di nuovi file o la modifica di file esistenti) vengono scritte su questo livello contenitore scrivibile

Docker container lifecycle



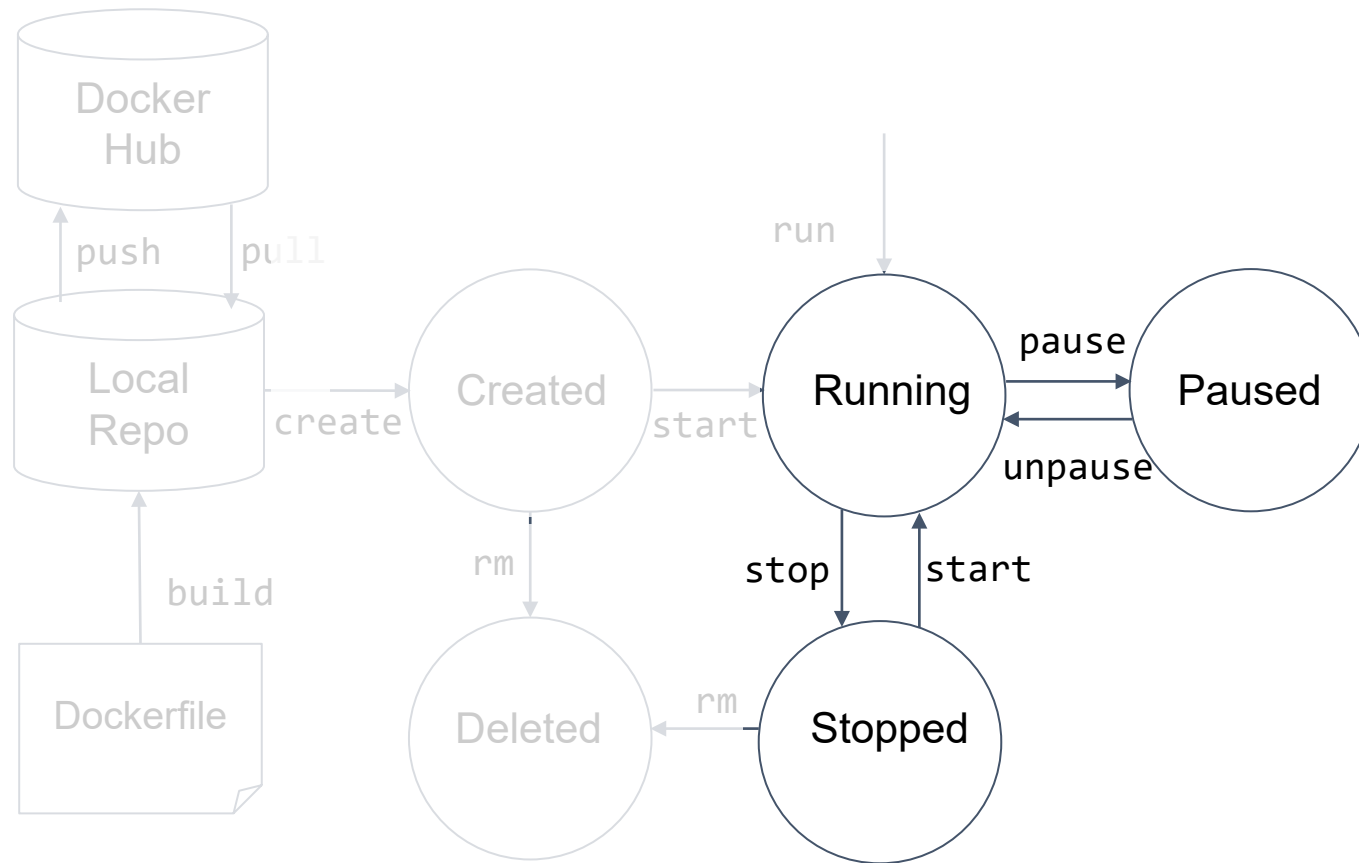
- Le immagini Docker possono essere estratte da un repository centrale (Docker Hub)
- Le immagini estratte vengono salvate in un repository locale
- Immagini personalizzate possono essere create e salvate a partire da uno specifico file di configurazione (Dockerfile)
- Le immagini personalizzate possono essere inserite nel repository centrale

Docker container lifecycle



- Un'immagine salvata può essere utilizzata per creare un contenitore (viene aggiunto un livello scrivibile)
- Avviare un contenitore significa eseguire un comando predefinito contenuto nell'immagine, ovvero l'entrypoint (può essere sovrascritto)
- Un contenitore può essere creato e avviato utilizzando un singolo comando di esecuzione

Docker container lifecycle



- Dopo aver eseguito l'entrypoint, il contenitore si ferma
- Un processo **in primo piano** specificato come punto di ingresso può continuare a eseguire il contenitore
- Un contenitore in esecuzione può essere messo in pausa o arrestato

Esempio del ciclo di vita di Docker: immagini

- Estrai un'immagine dall'hub centrale
 - `docker pull <immagine>`
- Crea un'immagine da un Dockerfile
 - `docker build -t <immagine>`
- Elenca le immagini salvate nel repository locale
 - `docker images`
- Elimina un'immagine
 - `docker rmi <immagine>`

Esempio del ciclo di vita di Docker: contenitori

- Start a container
 - `docker run <image>`
- Start a container overriding default entrypoint with `<newcmd>`
 - `docker run --entrypoint <newcmd> <image>`
- Execute a command `<cmd>` (e.g., `/bin/bash`) in a running container
 - `docker exec -it <containerID> <cmd>`
- Stop a container
 - `docker stop <containerID>`
- Remove a container
 - `docker rm <containerID>`
- List running and stopped containers
 - `docker ps -a`

Volumi Docker

- **I volumi possono essere utilizzati per salvare (persistere) i dati e per condividere dati tra contenitori**
- **non è correlato ai livelli del contenitore: l'eliminazione di un contenitore non implica l'eliminazione di un volume associato**
- **Un volume può essere:**
 - (anonimo/)nominato: gestito internamente da Docker stessa
 - host: si riferisce alla posizione del file system dell'host che esegue Docker

Dockerfile

- Docker può creare automaticamente immagini personalizzate leggendo le istruzioni da un Dockerfile
- Dockerfile è un documento di testo che contiene tutti i comandi che un utente può richiamare sulla riga di comando per assemblare un'immagine
- La *docker build* permette l'esecuzione di una build automatizzata di un'immagine a partire da un Dockerfile

(Riferimento Dockerfile :
<https://docs.docker.com/engine/reference/builder/>)

Esempio di

Comando

Descrizione Esempio

FROM <image>

Start building from this (base) image

FROM ubuntu

RUN <cmd>

Run the specified command

RUN apt install apache2

COPY <src> <dest>

Copy a file to the image fs

COPY vh.conf /etc/apache2/conf/

CMD ["exec", "param1", ...]

Configure the default command when container starts

CMD ["apache2", "-D", "FOREGROUND "]

Docker compone

Compose è uno strumento per definire ed eseguire applicazioni Docker multi-contenitore

- Un unico file per fornire le configurazioni
- Un unico set di comandi per configurare, creare ed eseguire tutti i contenitori

Docker compone la configurazione

- Il file Compose (docker- compose.yaml) utilizza una sintassi standard leggibile dall'uomo, ovvero la sintassi YAML *. Definisce
 - Servizi: configurazione applicata a ciascun contenitore (molto simile al passaggio dei parametri della riga di comando a *docker run*)
 - Reti (facoltativo): definire la configurazione delle reti da creare
 - Volumi (facoltativo): definire la configurazione dei volumi da creare

* <https://yaml.org/>

Docker compose: esempio di comandi

- (creare immagini e) eseguire servizi
 - `docker-compose up -d`
- interrompere i servizi
 - `docker-compose stop`
- avviare i servizi
 - `docker-compose start -d`
- fermare e rimuovere contenitori e reti
 - `docker-compose down`
- mostra i log (output del punto di ingresso)
 - `docker-compose logs -f [ nome_servizio ]`
- (build images and) run services
 - `docker-compose up -d`
- stop services
 - `docker-compose stop`
- start services
 - `docker-compose start -d`
- stop and remove containers and networks
 - `docker-compose down`
- show logs (entrypoint output)
 - `docker-compose logs -f [service_name]`